

Paimon Spark-3.3 核心架构设计文档

核心架构设计 · 设计模式 · 流程图 · 时序图 · 调用链

模块：paimon-spark-3.3

分支：release-1.3.1

生成工具：AI Architecture Analyzer

生成日期：2026年03月27日

第1章 模块概述

1.1 模块定位

paimon-spark-3.3 是 Apache Paimon 与 Apache Spark 3.3 版本的适配层模块。该模块的核心职责是将 Paimon 数据湖存储引擎无缝集成到 Spark 3.3 的 DataSource V2 框架中，使用户可以通过标准 Spark SQL 语法操作 Paimon 表，包括读取、写入、MERGE INTO、INSERT OVERWRITE、DDL 操作以及 View 管理等功能。

由于 Spark 3.3 相较于更高版本 (3.4/3.5/4.0) 在 API 层面存在较多差异，该模块通过 Java 占位接口、Scala Shim 层、以及版本特定的适配代码来弥合这些差异，确保公共模块 (paimon-spark-common) 的核心逻辑可以在 Spark 3.3 环境下正确运行。

1.2 核心目标

- 版本兼容适配：提供 Spark 3.3 缺失的 API 占位实现，如 TableCatalogCapability 枚举、SupportsReportOrdering 接口
- 完整 MERGE INTO 支持：通过 MergeRows 逻辑计划和 MergeRowsExec 物理执行计划实现行级合并操作
- 桶分区扫描：在 PaimonScan 中实现基于 HASH_FIXED 模式的桶分区感知读取
- CTAS 执行策略：通过 PaimonCreateTableAsSelectStrategy 处理 CREATE TABLE AS SELECT
- View 管理：通过 RewritePaimonViewCommands 重写 CREATE/DROP/SHOW VIEW 命令
- 标量子查询优化：通过 MergePaimonScalarSubqueries 合并相同 Paimon 表的标量子查询

1.3 模块统计

指标	数值
主要编程语言	Scala (95.3%)
Java 源文件	2 个
Scala 主源码文件	18 个
Scala 测试文件	23 个
总文件数	43 个

1.4 源码文件清单

文件名	包路径	职责描述
TableCatalogCapability.java	o.a.spark.sql.connector.catalog	TableCatalog 能力枚举占位

文件名	包路径	职责描述
SupportsReportOrdering.java	o.a.spark.sql.connector.read	排序报告接口编译占位
PaimonScan.scala	o.a.paimon.spark	核心扫描实现，支持桶分区
SparkTable.scala	o.a.paimon.spark	Paimon 表的 Spark Table 包装
OldCompatibleStrategy.scala	o.a.paimon.spark.execution	MergeRows 物理计划策略转换
Compatibility.scala	o.a.paimon.spark.catalyst	版本兼容工具方法集
PaimonSupportsRuntimeFiltering.scala	o.a.paimon.spark.read	运行时分区过滤支持
SQLConfUtils.scala	o.a.paimon.spark.util	SQL 配置工具
PaimonMergeInto.scala	o.a.paimon.spark.catalyst.analysis	MergeInto 后处理解析规则
PaimonMergeIntoResolver.scala	o.a.paimon.spark.catalyst.analysis	MergeInto 条件动作解析器
MergePaimonScalarSubqueries.scala	o.a.paimon.spark.catalyst.optimizer	标量子查询合并优化器
TypeUtils.scala	o.a.paimon.spark.util.shim	类型转换 Shim (timestamp_ntz)
PaimonCreateTableAsSelectStrategy.scala	o.a.spark.sql.execution.shim	CTAS 执行策略
PredicateUtils.scala	o.a.spark.sql.internal.connector	V2 Predicate 转 V1 Filter
MinorVersionShim.scala	o.a.spark.sql.paimon.shims	Spark 小版本兼容适配
RewritePaimonFunctionCommands.scala	o.a.spark.sql.catalyst.parser.extensions	函数命令重写 (空操作)
RewritePaimonViewCommands.scala	o.a.spark.sql.catalyst.parser.extensions	View DDL 命令重写
MergeRows.scala	o.a.spark.sql.catalyst.plans.logical	MERGE INTO 逻辑计划节点
MergeRowsExec.scala	o.a.spark.sql.execution.datasources.v2	MERGE INTO 物理执行计划

第2章 核心架构设计

2.1 整体分层架构

paimon-spark-3.3 模块采用经典的四层架构设计，自上而下分别是：SQL 解析扩展层、Catalyst 分析/优化层、物理执行策略层和基础适配层。各层通过 Spark 内部扩展机制（SparkSessionExtensions）有机串联。

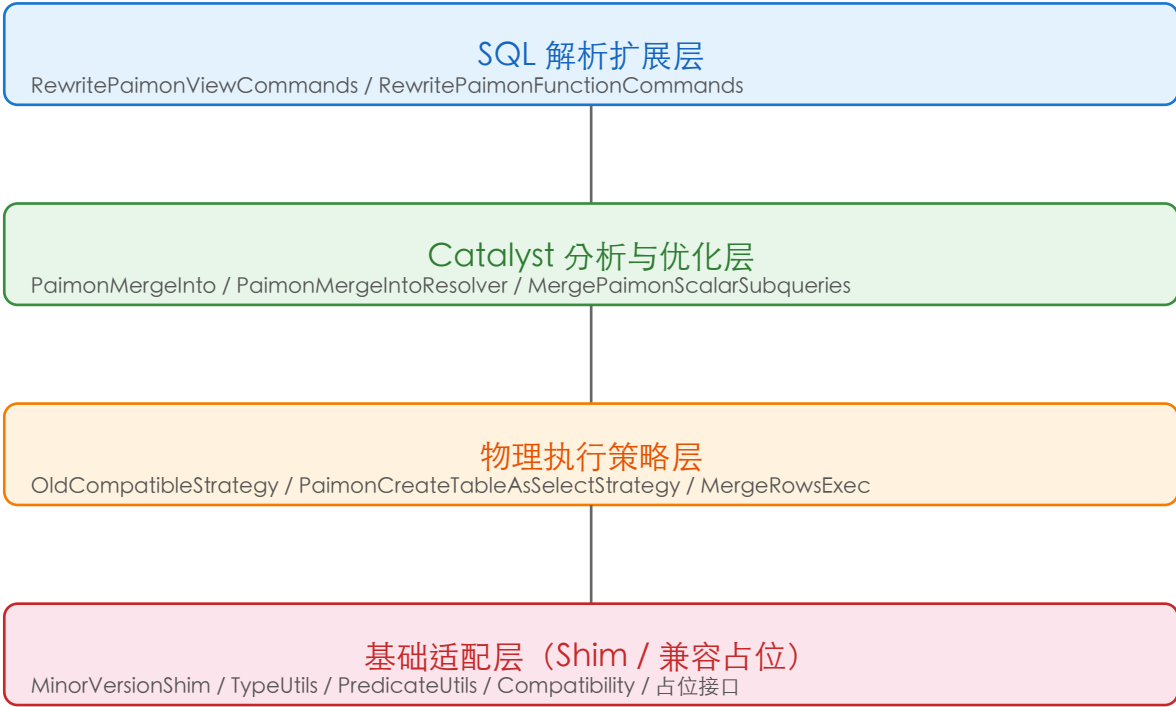


图2-1 paimon-spark-3.3 整体分层架构图

2.2 类继承体系

模块中的核心类通过继承 paimon-spark-common 公共模块的基类来实现版本特定适配。以下是主要继承关系：

本模块类	继承自（公共模块）	混入特质
PaimonScan	PaimonBaseScan	SupportsReportPartitioning
SparkTable	PaimonSparkTableBase	-
PaimonMergeInto	PaimonMergeIntoBase	-
PaimonMergeIntoResolver	PaimonMergeIntoResolverBase	-
MergePaimonScalarSubqueries	MergePaimonScalarSubqueriesBase	-

本模块类	继承自（公共模块）	混入特质
OldCompatibleStrategy	SparkStrategy	PredicateHelper, PaimonLookupCatalog
PaimonCreateTableAsSelectStrategy	Strategy	PaimonStrategyHelper
MergeRows	UnaryNode (LogicalPlan)	-
MergeRowsExec	UnaryExecNode (SparkPlan)	-
PaimonSupportsRuntimeFiltering	BaseScan	SupportsRuntimeFiltering

2.3 Spark 3.3 版本限制与适配方案

Spark 3.3 相较于更高版本缺少以下特性，本模块均有对应适配方案：

缺失特性	适配方案	实现文件
TableCatalogCapability 枚举	提供 Java 枚举占位类	TableCatalogCapability.java
SupportsReportOrdering 接口	提供空接口，仅编译通过	SupportsReportOrdering.java
timestamp_ntz 类型	TypeUtils 将 Paimon TimestampType 映射为 Spark TimestampType	TypeUtils.scala
WHEN NOT MATCHED BY SOURCE	PaimonMergeIntoResolver 返回 Seq.empty	PaimonMergeIntoResolver.scala
V2 Predicate 转 V1 Filter	从 Spark 3.4+ 反向移植 PredicateUtils	PredicateUtils.scala
CTERelationRef 参数差异	MinorVersionShim 省略 isStreaming 参数	MinorVersionShim.scala
MergeIntoTable 参数差异	MinorVersionShim 省略 notMatchedBySource 参数	MinorVersionShim.scala
DEFAULT 列语法	测试中跳过 DEFAULT 相关断言	DescribeTableTest.scala
Function Commands 重写	RewritePaimonFunctionCommands 为空操作	RewritePaimonFunctionCommands.scala

第3章 核心类设计模式详解

3.1 PaimonScan — 模板方法 (Template Method) + 策略 (Strategy) 模式

PaimonScan

继承

PaimonBaseScan，采用模板方法模式：基类定义了扫描的通用骨架流程（初始化 ReadBuilder、获取 InputSplits、读取 Schema 等），子类 PaimonScan 覆写 outputPartitioning() 和 getInputPartitions() 提供版本特定的桶分区策略。

策略选择逻辑：

- 当表为 HASH_FIXED 桶模式、单桶键、且 Spark v2BucketingEnabled 时，启用 KeyGroupedPartitioning 策略
- 否则退化为 UnknownPartitioning，由基类默认实现处理

关键方法：

- extractBucketTransform — 从 FileStoreTable 中提取桶分区 Transform（懒加载）
- extractBucketNumber() — 从 DataSplit 中提取统一的桶总数
- shouldDoBucketedScan — 综合判断是否执行桶分区扫描
- outputPartitioning() — 返回 KeyGroupedPartitioning 或 UnknownPartitioning
- getInputPartitions(splits) — 按 bucket 分组生成 PaimonBucketedInputPartition

3.2 MergeRows / MergeRowsExec — 命令 (Command) + 迭代器 (Iterator) 模式

MergeRows 作为逻辑计划节点采用命令模式，定义三类指令 (Instruction)：Keep（保留行）、Discard（丢弃行）、Split（拆分行）。每条指令封装了条件表达式和输出投影，构成独立的可执行命令。

MergeRowsExec

作为物理执行节点，内部通过

MergeRowIterator

实现迭代器模式。迭代器逐行处理 JOIN 结果，根据源行/目标行的存在性分派到对应指令序列，按优先级顺序匹配并执行第一条满足条件的指令。

指令类型：

- Keep(condition, output) — 条件匹配时投影输出行
- Discard(condition) — 条件匹配时丢弃行（返回 null）
- Split(condition, output, otherOutput) — 条件匹配时拆分为两行：主输出行 + 缓存额外行

3.3 OldCompatibleStrategy — 策略 (Strategy) + 工厂方法 (Factory Method) 模式

OldCompatibleStrategy 继承 SparkStrategy，实现 Spark 的 Strategy 接口。其 apply() 方法作为工厂方法，通过模式匹配将 MergeRows 逻辑计划节点转换为 MergeRowsExec 物理执行计划节点。

关键方法：

- apply(plan: LogicalPlan) — 匹配 MergeRows 并构造 MergeRowsExec

3.4 PaimonCreateTableAsSelectStrategy — 策略 (Strategy) 模式

PaimonCreateTableAsSelectStrategy 实现 Spark 的 Strategy 接口，专门处理 CTAS 语句。核心逻辑是：从 options 中分离 CoreOptions 和 IcebergOptions 作为表属性，剩余作为写入选项，然后构造 CreateTableAsSelectExec 执行节点。同时检测并拒绝分区 Format 表的 CTAS 操作。

3.5 RewritePaimonViewCommands — 规则 (Rule) + 访问者 (Visitor) 模式

RewritePaimonViewCommands 继承 Rule[LogicalPlan]，采用 Spark 的规则模式。其 apply() 方法通过 resolveOperatorsUp 遍历逻辑计划树（访问者模式），将 Spark 原生的 CreateView、DropView、ShowViews 命令转换为 Paimon 专属的 CreatePaimonView、DropPaimonView、ShowPaimonViews 计划节点。

关键方法：

- apply(plan) — 遍历逻辑计划树并重写 View 命令
- ResolvedIdent.unapply() — 提取器对象，识别 Paimon Catalog 的 View 标识符
- isTempView() — 检查是否为临时视图（临时视图不重写）

3.6 PredicateUtils — 适配器 (Adapter) 模式

PredicateUtils 是从 Spark 3.4+ 反向移植的谓词转换工具，实现适配器模式。将 Spark V2 的 Predicate 接口转换为 V1 的 Filter 接口，支持 17 种谓词类型的转换 (IN、=、<>、>、<、>=、<=、IS_NULL、IS_NOT_NULL、STARTS_WITH、ENDS_WITH、CONTAINS、AND、OR、NOT、ALWAYS_TRUE、ALWAYS_FALSE)。

3.7 PaimonSupportsRuntimeFiltering — 观察者 (Observer) + 模板方法模式

PaimonSupportsRuntimeFiltering 实现 SupportsRuntimeFiltering 接口，采用观察者模式响应运行时过滤事件。当 Spark 引擎推送运行时 Filter 时，filter() 方法被调用，将过滤条件转换为 PartitionPredicate 并应用到 ReadBuilder，同时清空缓存的 inputSplits 和 inputPartitions 以触发重新计算。

关键方法：

- filterAttributes() — 返回可接受的运行时过滤属性（分区键交集）

- filter(filters) — 接收运行时 Filter，转换并追加到分区过滤条件
- inputSplits / inputPartitions — 懒加载，filter 后置空触发重算

第4章 核心流程图

4.1 PaimonScan 桶分区扫描流程

PaimonScan 的桶分区扫描流程是本模块数据读取的核心路径，决定了是否能利用桶分区信息避免全量 Shuffle。

Spark

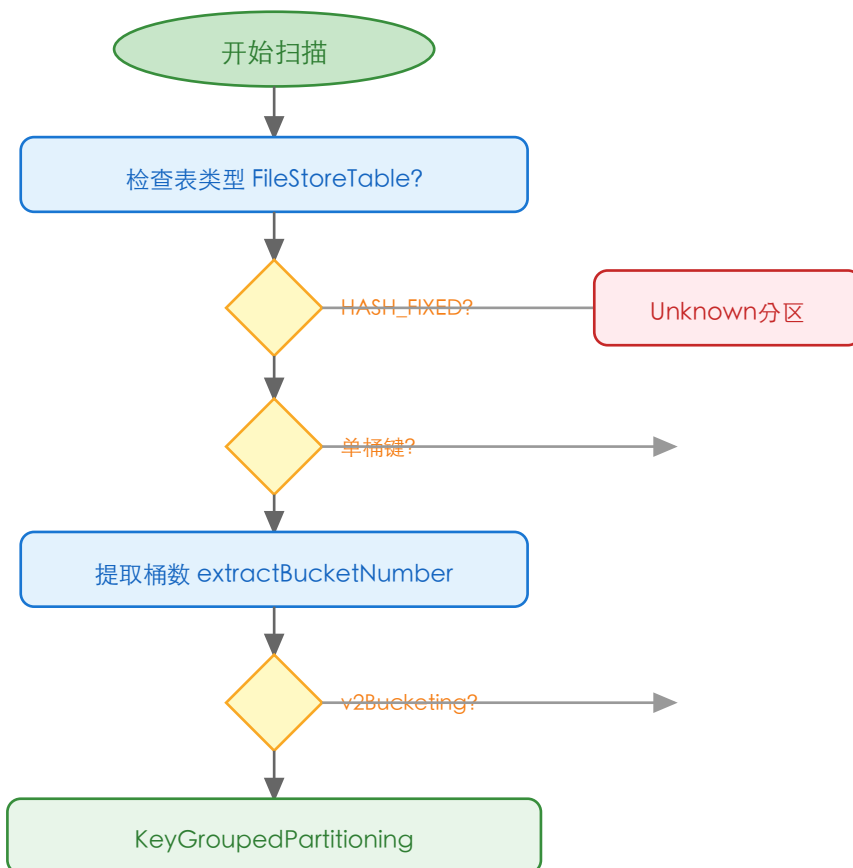


图4-1 PaimonScan 桶分区扫描决策流程图

4.2 MERGE INTO 执行流程

MERGE INTO 操作是本模块最复杂的功能之一，经历从 SQL 解析到物理执行的完整 Spark Catalyst 管线。

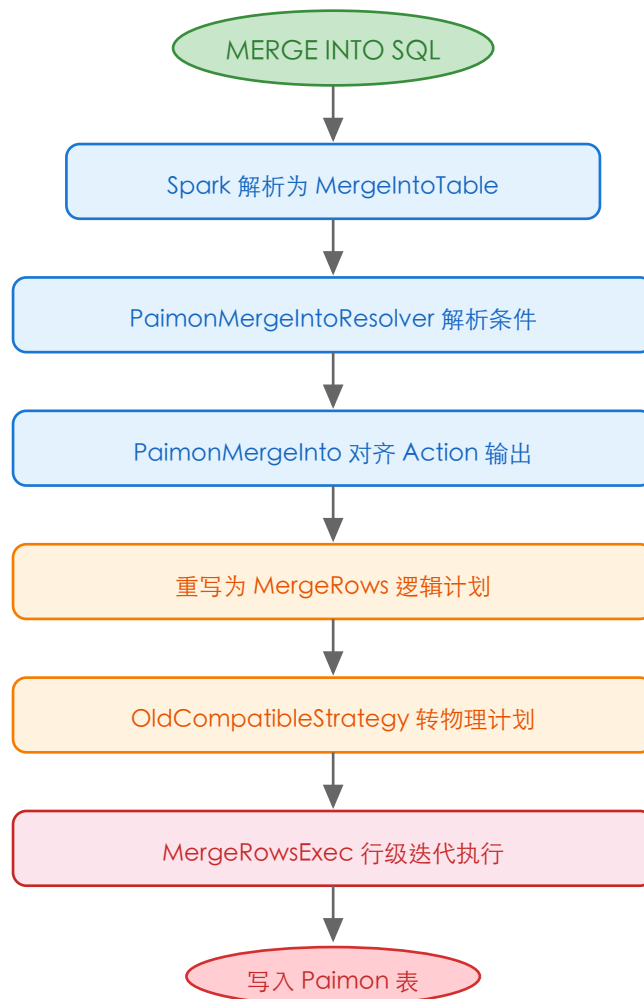


图4-2 MERGE INTO 完整执行流程图

4.3 RewritePaimonViewCommands 视图命令重写流程

View 管理功能通过 SQL 解析扩展层拦截 Spark 原生命令并重写为 Paimon 专属逻辑计划节点。

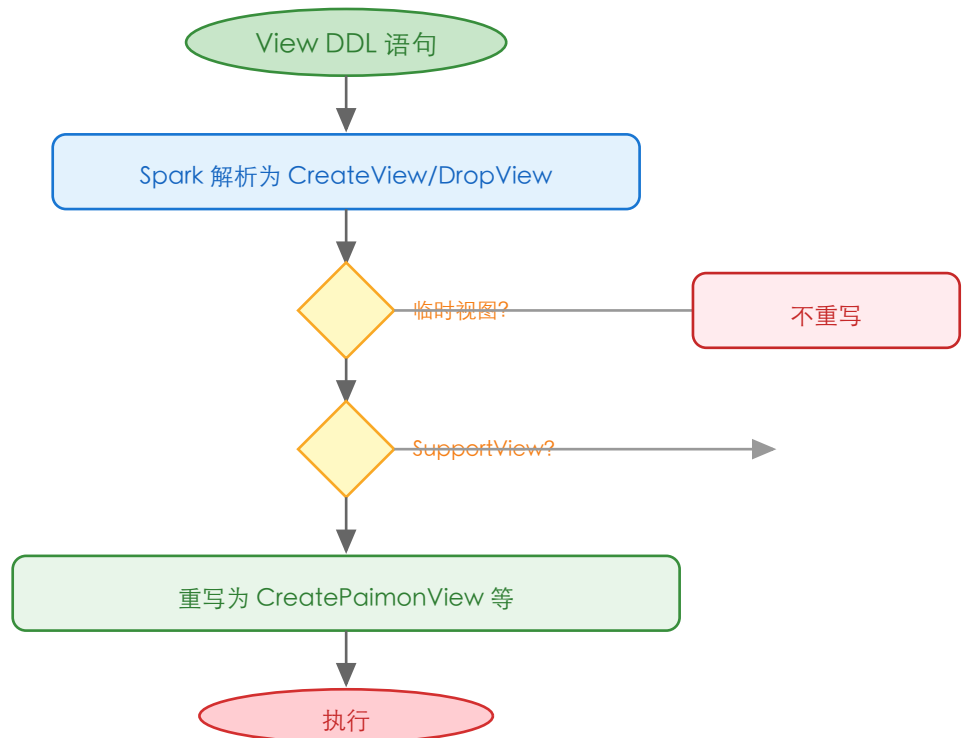


图4-3 View 命令重写流程图

第5章 核心时序图

5.1 MERGE INTO 行级处理时序图

该时序图展示了 MergeRowsExec 内部 MergeRowIterator 处理每一行的交互流程：

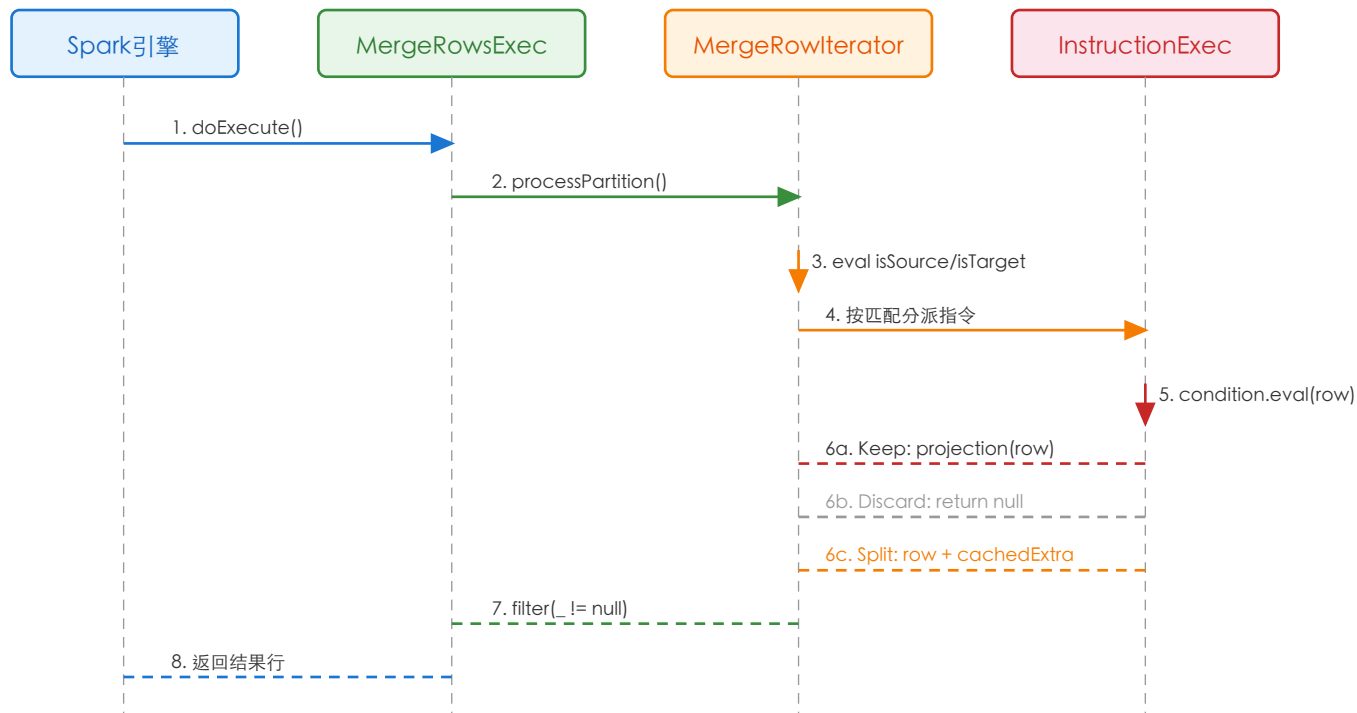


图5-1 MERGE INTO 行级处理时序图

5.2 CTAS (CREATE TABLE AS SELECT) 执行时序图

展示 PaimonCreateTableAsSelectStrategy 如何处理 CTAS 语句：

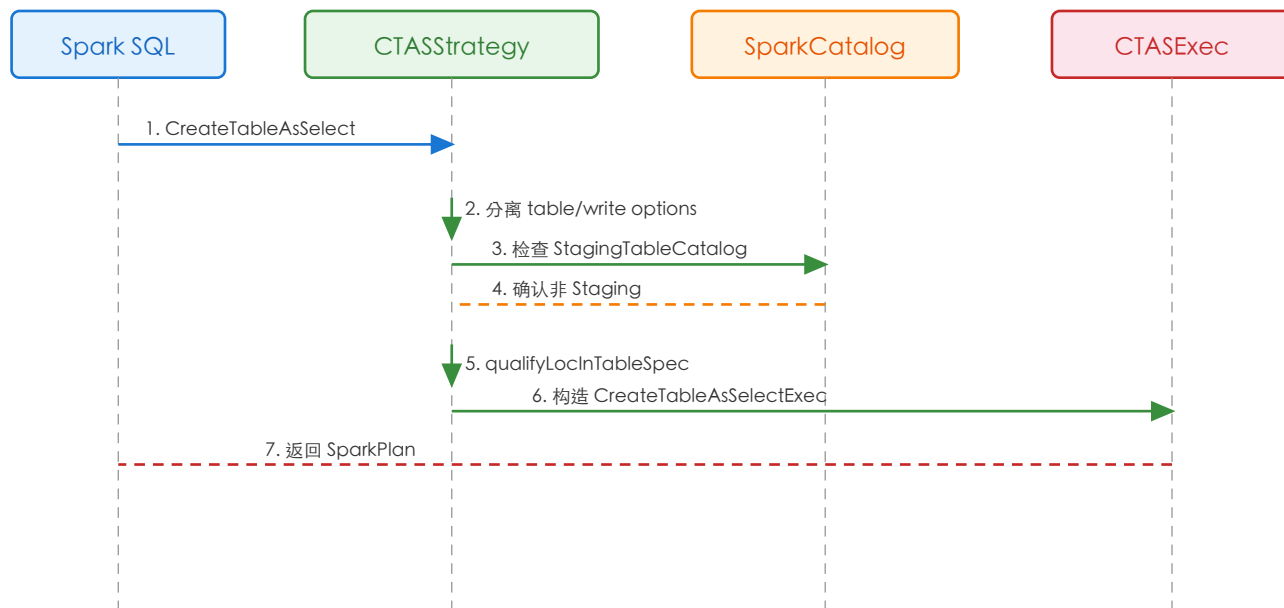


图5-2 CTAS 执行时序图

5.3 运行时过滤时序图

展示 PaimonSupportsRuntimeFiltering 如何在扫描过程中接受运行时推送的过滤条件：

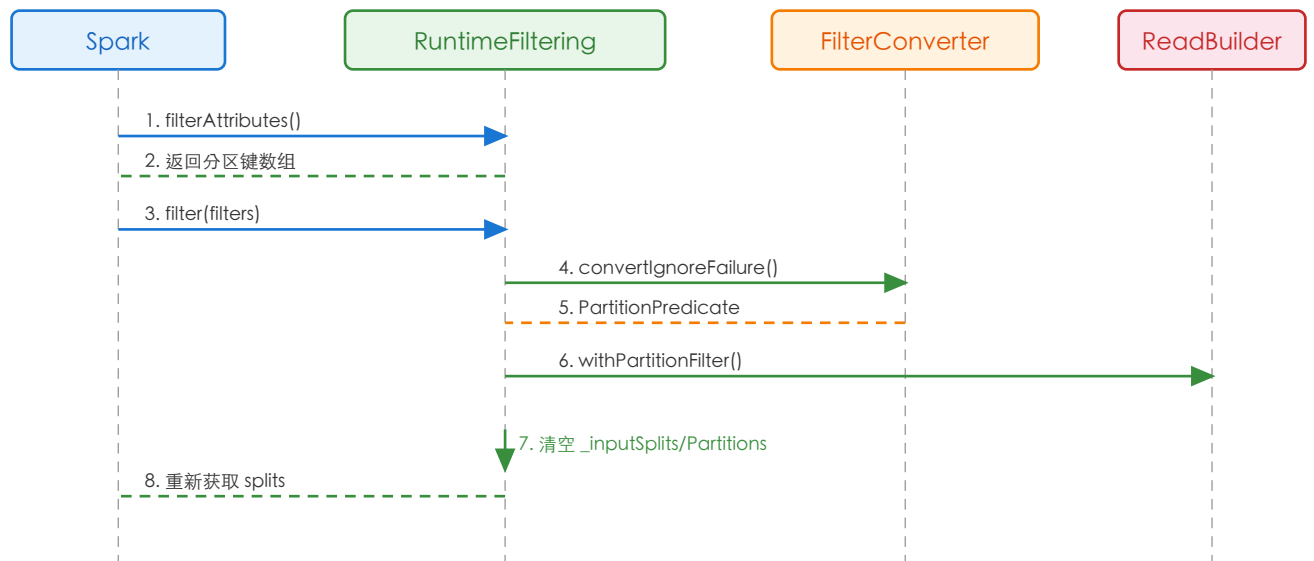


图5-3 运行时过滤时序图

第6章 关键场景调用链

6.1 场景一：桶分区感知扫描

触发条件：用户执行 SELECT 查询且表配置为 HASH_FIXED 桶模式、单桶键、Spark 开启 v2Bucketing

调用链路：

步骤	类.方法	说明
1	PaimonScan.outputPartitioning()	Spark 调用获取分区方案
2	PaimonScan.extractBucketTransform	懒加载，检查 FileStoreTable
3	FileStoreTable.bucketSpec()	获取桶规格（模式、桶数、桶键）
4	PaimonScan.extractBucketNumber()	从 DataSplit 提取统一桶总数
5	PaimonScan.shouldDoBucketedScan	综合判断: !disabled && v2Bucketing && transform.isDefined
6	Expressions.bucket(num, key)	构造 Spark Bucket Transform
7	new KeyGroupedPartitioning()	创建键分组分区方案
8	PaimonScan.getInputPartitions()	按 bucket 分组生成 PaimonBucketedInputPartition

6.2 场景二：MERGE INTO 完整执行

触发条件：用户执行 MERGE INTO 语句合并源表数据到目标 Paimon 表

调用链路：

步骤	类.方法	说明
1	PaimonMergeIntoResolver.apply()	解析 MergeIntoTable 中的条件和 Action
2	PaimonMergeIntoResolver.build()	构建解析后的 MergeIntoTable (Spark 3.3 不支持 notMatchedBySource)
3	PaimonMergeInto.apply()	对齐 matchedActions 和 notMatchedActions 的输出属性
4	PaimonMergeInto.alignMergeAction()	按目标表 output 重新排列 Action 的输出列
5	Catalyst -> MergeRows	Spark 将 MergeIntoTable 重写为 MergeRows 逻辑计划

步骤	类.方法	说明
6	OldCompatibleStrategy.apply()	匹配 MergeRows 并创建 MergeRowsExec
7	MergeRowsExec.doExecute()	分区级执行：创建谓词、投影、基数校验器
8	MergeRowIterator.next()	逐行处理：判断 source/target 存在性，分派指令
9	InstructionExec (Keep/Discard/Split)	按条件执行保留/丢弃/拆分操作
10	BitmapCardinalityValidator.validate()	使用 Roaring64Bitmap 校验目标行唯一匹配

6.3 场景三：CREATE TABLE AS SELECT (CTAS)

触发条件：用户执行 CREATE TABLE ... AS SELECT 语句

调用链路：

步骤	类.方法	说明
1	PaimonCreateTableAsSelectStrategy.apply()	匹配 CreateTableAsSelect 逻辑计划
2	CoreOptions.getOptions()	获取所有核心配置键列表
3	IcebergOptions.getOptions()	获取 Iceberg 兼容配置键列表
4	options.partition()	将 options 分为表属性和写入选项
5	catalog.isFormatTable()	检查是否为分区 Format 表（不支持 CTAS）
6	qualifyLocInTableSpec()	标准化表规格中的 location 路径
7	CreateTableAsSelectExec()	构造物理执行节点

6.4 场景四：View 管理（CREATE VIEW）

触发条件：用户执行 CREATE VIEW 语句且当前 Catalog 支持 View

调用链路：

步骤	类.方法	说明
1	RewritePaimonViewCommands.apply()	遍历逻辑计划树
2	ResolvedIdent.unapply()	从 UnresolvedDBObjectName 提取 SupportView Catalog
3	isTempView()	排除临时视图
4	CTESubstitution.apply(query)	对查询进行 CTE 替换
5	CreatePaimonView()	构造 Paimon 专属的 View 创建计划节点

6.5 场景五：V2 谓词转 V1 Filter

触发条件：Spark 引擎通过 V2 Predicate 接口下推过滤条件时需要转换为 V1 Filter

调用链路：

步骤	类.方法	说明
1	PredicateUtils.toV1(predicate)	入口方法，接收 V2 Predicate
2	predicate.name() match	按谓词名称分派：IN/=/<>/>/</IS_NULL 等
3	isValidBinaryPredicate()	校验二元谓词结构（NamedRef + LiteralValue）
4	CatalystTypeConverters.convertToScala()	将 V2 字面量值转为 Scala 类型
5	构造 V1 Filter	返回 EqualTo/GreaterThan/In/IsNull/And/Or/Not 等

6.6 场景六：标量子查询合并优化

触发条件：查询中包含多个引用同一 Paimon 表的标量子查询

调用链路：

步骤	类.方法	说明
1	MergePaimonScalarSubqueries.apply()	优化器规则入口
2	tryMergeDataSourceV2ScanRelation()	尝试合并两个 DataSourceV2ScanRelation
3	checkIdenticalPlans()	检查两个 Relation 的底层计划是否相同
4	samePartitioning()	检查分区方案是否兼容
5	mergePaimonScan()	合并两个 PaimonScan 的读取 Schema
6	createScalarSubquery()	构造合并后的 ScalarSubquery (Spark 3.3 签名)

第7章 配置与扩展

7.1 相关配置属性

配置项	来源	说明
spark.sql.sources.v2.bucketing.enabled	Spark SQLConf	控制是否启用 V2 桶分区扫描
spark.sql.sources.partitionOverwriteMode	Spark SQLConf	控制 INSERT OVERWRITE 的分区覆写模式 (static/dynamic)
spark.sql.catalog.paimon.cache-enabled	Paimon Catalog	控制 Paimon Catalog 是否启用缓存
primary-key	Paimon CoreOptions	表的主键字段列表
bucket	Paimon CoreOptions	桶数配置 (-1 表示动态桶)
bucket-key	Paimon CoreOptions	桶键字段列表
write-only	Paimon CoreOptions	仅写入模式 (Append 表无主键)

7.2 扩展点

模块提供以下扩展点供 Spark 版本适配：

- MinorVersionShim — Spark 小版本间 API 差异的适配层，可为其他 Spark 3.3.x 子版本定制
- TypeUtils — 类型映射策略的扩展点，当前将 timestamp_ntz 映射为 timestamp
- Compatibility — Catalyst 表达式兼容方法集，可覆写 cast、withNewQuery 等
- PaimonBaseScan / PaimonSparkTableBase — 公共模块基类，版本适配模块可覆写特定方法

第8章 设计亮点与总结

8.1 设计亮点

- 优雅的版本适配分层：通过 Java 占位类 + Scala Shim 层 + 公共基类覆写三层机制，将 95% 以上的逻辑保留在公共模块，版本适配模块仅包含必要的差异代码，极大降低了维护成本。
- 完整的 MERGE INTO 实现：从 MergeRows 逻辑计划到 MergeRowsExec 物理执行的三层设计（逻辑→策略→物理），支持 Keep/Discard/Split 三种指令，涵盖了行级合并的所有场景。BitmapCardinalityValidator 使用 Roaring64Bitmap 实现高效的唯一匹配校验。
- 精准的桶分区感知：PaimonScan 通过 extractBucketTransform 和 shouldDoBucketedScan 的精细判断逻辑，在满足条件时启用 KeyGroupedPartitioning，显著减少不必要的 Shuffle 操作。
- 反向移植策略：PredicateUtils 从 Spark 3.4+ 完整移植，保持 API 签名一致性，为后续版本升级提供平滑迁移路径。
- 测试复用架构：23 个测试类中绝大多数为空壳类，继承公共模块的 TestBase 基类，仅在必要时（如 DescribeTableTest 跳过 DEFAULT 语法）提供版本特定的定制。

8.2 设计限制

- 不支持 WHEN NOT MATCHED BY SOURCE：Spark 3.3 原生不支持此子句，PaimonMergeIntoResolver 直接返回空序列，若传入则抛出 RuntimeException。
- 不支持 timestamp_ntz 类型：所有 Paimon TimestampType 统一映射为 Spark TimestampType，可能在时区敏感场景产生语义差异。
- 单桶键限制：PaimonScan 的桶分区优化仅支持单桶键场景，多桶键表将退化为 UnknownPartitioning。
- 不支持分区 Format 表 CTAS：PaimonCreateTableAsSelectStrategy 显式抛出 UnsupportedOperationException。

8.3 质量评估

评估维度	评分	说明
代码结构	★★★★★	清晰的分层设计，公共模块与版本适配模块职责分明
设计模式运用	★★★★☆	恰当运用模板方法、策略、命令、适配器等模式，无过度设计
可扩展性	★★★★☆	Shim 层设计使新版本适配成本极低，扩展点明确
版本兼容性	★★★★★	全面覆盖 Spark 3.3 的 API 差异，编译占位精简高效
测试覆盖	★★★★☆	测试复用架构优秀，版本特定测试有针对性

8.4 总结

paimon-spark-3.3 模块是 Apache Paimon 多版本 Spark 支持策略的典范实现。通过精心设计的分层架构和版本适配机制，该模块以仅 43 个源文件（20 个主源码 + 23 个测试）的精简规模，实现了对 Spark 3.3 DataSource V2 的完整集成，包括桶分区扫描、MERGE INTO 行级合并、CTAS 执行、View 管理、运行时过滤和标量子查询优化等核心功能。其设计理念——将通用逻辑上提到公共模块、将差异代码下沉到版本适配模块——值得在类似的多版本兼容项目中借鉴。